Midterm Report

BC Personal Health Management Platform

An AI-Powered Healthcare Navigation and Analytics System

CSIS 4495 - 002: Applied Research Project

Winter 2026

Prepared For:

Padmapriya Arasanipalai Kandhadai

Date: February 14, 2026

Github Link: https://github.com/desporamon/W26_4495_S2_DesmondC

Video Demo: https://youtu.be/ewBCIXlxzsI

Prepared By:

Name	Student ID	Email Address
Desmond Chua	300369803	chuad1@student.douglascollege.ca

Contents

1. Introduction

1.1 Domain Background

Residents in British Columbia face a lot of hurdles when it comes to healthcare navigation and acquiring medical services on time. The challenge lies in the fact that the province of BC with a population of over 5 million people is served by a complex network of hospitals along with urgent care centers, walk-in clinics, and virtual care options. The issue of deciding which level of treatment is suitable for what kind of symptoms can get extremely confusing mainly for new migrants and people who are not familiar with the health system of the province. Long waiting times in emergency departments have become one of the most visible signs of the crisis as they are increasing the problem by patients that arrive with non-urgent complaints and cannot be treated in primary care setting or through self-care. In fact, the average waiting time in emergency departments was 3.1 hours across the whole BC in 2023.

The Canadian Triage and Acuity Scale (CTAS), developed by the Canadian Association of Emergency Physicians, is a model which helps to standardize patient urgency assessment in emergency departments. However, this clinical tool is not yet open for the general public. There is a disconnect between the clinical understanding that is in triage frameworks and the information patients have available to them when they decide how to seek care. Hence, this gap is a problem that needs to be resolved in healthcare delivery but at the same time, it is a big opening for applied data analytics.

1.2 Problem Statement and Objectives

The main problem this research is trying to solve is the absence of an accessible, smart tool that could help the residents of BC make well-informed healthcare decisions even before they get to a healthcare facility. In fact, the symptom-checking tools that are available online these days are generally quite generic, not specifically designed for the BC healthcare system, and most of them do not have the necessary safety measures to handle medical emergencies.

The main objectives of this initiative are:

(1) to design an AI symptom checker that leverages natural language processing and machine learning classification to deliver recommendations on the level of urgency of the situation;

(2) to set up a deterministic safety layer guided by CTAS standards that can effectively recognize situations that are life-threatening;

(3) to develop a user-oriented healthcare platform with features such as personal health diary, locating healthcare facilities, and providing educational material; and

(4) to use the project as a case study for demonstrating the use of data analytics methods like classification, clustering, and anomaly detection in healthcare.

1.3 Review of Related Literature

Azeez et al. (2015) were able to achieve an intelligent triage system by using an ensemble Random Forest technique at the Universiti Kebangsaan Malaysia Medical Center. Their research deals with the problem of class imbalance by using randomized resampling, thus resulting in an out-of- bag error of 0.02 with 300% resampling when compared to 0.37 without any preprocessing. Such a finding is very significant in the case of healthcare datasets where some urgency levels may have only a few examples.

Holcomb et al. (2014) validated the same Random Forest Model for pre-hospital trauma triage at the University of Texas Level I Trauma Center, thus proving the model capable of being used for clinical decision support in real time scenarios. In more recent times, Kwon et al. (2018) implemented deep learning technologies to predict KTAS (Korean Triage and Acuity Scale) levels using 11,656 cases, thus resulting in a highly efficient classification model. All the above works provide a solid basis for the adoption of ML- based methods for triage classification systems.

In healthcare anomaly detection, Fernandes et al. (2020) and Esfandiari et al. (2021) demonstrated the applicability of Isolation Forest to detect rare patterns in clinical data. Vrbaski et al. (2019) resorted to K- Means clustering to profile patients by symptoms and healthcare usage patterns. The results of these works justify the multi-algorithm methodology of this work.

1.4 Hypotheses and Expected Benefits

The current program tests the assumption that a hybrid system, which combines rule-based safety checks with ML classification, can yield reliable urgency assessments while at the same time keeping patient safety intact. Among the expected benefits of the program: fewer unnecessary visits to the ED due to better self-triage instructions; quicker delivery of suitable care through recommendations for BC- specific facilities; and, an uplift of health literacy by means of an in-built integrated education library.

The confidence threshold of 70% in the ML layer is a measure to safeguard that cases which are uncertain are conscientiously managed whereby they revert to general OpenAI guidance instead of potentially incorrect classifications being handed out.

2. Summary of Initially Proposed Research Project

The BC Personal Health Management Platform is an AI-powered web application to assist residents of British Colombia in making healthcare decisions. It is envisioned as a five-feature system developed with Streamlit (Python web framework) , and using Open AI GPT for natural language processing, scikit-learn for machine learning, and Power BI for data visualization.

The five features planned were:

(1) an **AI, Powered Symptom Assessment Chatbot** leveraging a 3-layer hybrid classification system (OpenAI NLP, CTAS rule, based safety checks, and ML classification);

(2) a **Personal Health Dashboard** to store and visualize individual assessment data;

(3) a **BC Healthcare Facility Finder** using map-based location services;

(4) a **Health Education Resource Library** with curated BC-specific content; and

(5) a **System Analytics Dashboard** for analyzing population-level trends. The whole platform was conceptually a **Streamlit** multi-page application taking SQLite for its data persistence layer and **bcrypt** for its authentication security.

Technically, the structure revolved around a 3-layer hybrid classification approach for the chatbot: **Layer 1** relies on OpenAI GPT to parse the natural language input and formulate structured symptom data; **Layer 2** implements a set of deterministic CTAS-based rules that are capable of detecting life-threatening emergencies; **Layer 3** utilizes a Random Forest ML classifier in identifying the urgency levels of non-critical cases. The justification for this arrangement can be found in various academic papers that demonstrate the high performance of Random Forest for triage classification (Goto et al. , 2019; Azeez et al. , 2015).

The overall timeline plan of the project was presented in four sequential phases: Phase 1 (January 13 to February 9) focused on environment setup and foundation building; Phase 2 (February 10 to March 9) was dedicated to developing the core chatbot and training the ML model; Phase 3 (March 10 to March 30) was assigned to dashboard creation and analytics; Phase 4 (March 31 to April 14) was the period for integration testing and final documentation. To better support an Agile framework, we later converted these phases into **sprints**, as shown in the updated timelines of Progress Report 1 and Gantt Chart shown in section 4.

3. Changes to the Proposal

Since the original proposal submission, the project scope and technology stack have gone through a number of changes that are mostly based on the practical implementation experience, instructor feedback, and technical feasibility assessments.

3.1 OpenAI Model Update

The proposal identified GPT-3.5-turbo as the OpenAI model to be used for extracting natural language symptoms. However, the implementation phase led to a system change where a newer GPT model was used to take advantage of better instruction, following features and more consistent JSON- structured output. Architecturally, OpenAI's primary role in the project has not been altered: it is the NLP translation layer (Layer 1) that takes free-text symptom descriptions and turns them into structured data for the rule-based and ML layers.

3.2 Dashboard Visualisation Approach

The initial plan included Power BI embedded dashboards for the Personal Dashboard (Feature 2) and System Analytics Dashboard (Feature 5). However, after considering the technical challenge of embedding Power BI dashboards in Streamlit, the choice was made to develop dashboards using Streamlit widgets and Plotly charting libraries only. The decision to switch was based on a few reasons: (a) embedding Power BI needs a Power BI Pro/Premium license and a quite complicated authentication integration; (b) Streamlit has its own charting features through st. plotly_chart, st. bar_chart, and st. line_chart that fit in perfectly with the existing Python codebase; and (c) creating dashboards natively enables direct access to real-time data updates from the SQLite database without the need for a separate data pipeline to Power BI.

This change does not simplify the project technically; it just shifts the focus to Python-based data visualization and Streamlit interactive widgets, which is actually more in line with the Data Analytics specialization focus of the CSIS program.

3.3 Addition of ML Algorithm Research Phase

After the proposal feedback session with the instructor, I decided to add a separate ML Algorithm Research phase (Sprint 2C. 0) that would come right before model training. The instructor pointed out that it was not only necessary to justify the algorithm selection by conducting academic research but also the selection should not be made by an arbitrary choice. I came up with a background research report that explained my reasons for choosing Random Forest (classification), K-Means (clustering), and Isolation Forest (anomaly detection), based on the peer-reviewed ML for healthcare literature. The research phase gave the project a more solid academic base and is described in Section 5. 3.

3.4 Timeline Adjustments

Sprint 2 (chatbot development) had to be shortened to accommodate an extra research phase and still keep the original Phase 2 timeline. All Sprint 2 tasks (2A through 2D) were done by February 13, 2026, well in advance of the midterm deadline due to a family trip during reading week. There have been no changes to the overall project milestone dates.

3.5 Summary of Changes

The following table consolidates the key changes made since the initial proposal, comparing the original plan with the updated implementation and the rationale behind each decision.

Category	Initial Proposal	Updated Implementation	Justification
OpenAI Model	GPT-3.5-turbo for NLP symptom extraction	GPT-4o-mini with structured JSON output	Cost-effective choice: gpt-4o-mini offers better instruction-following and JSON parsing reliability than gpt-3.5-turbo at lower API cost, which is critical for a student-budget project with repeated API calls. The task (structured symptom extraction) does not require frontier reasoning models like GPT-5, making gpt-4o-mini the optimal balance of capability, speed, and cost.
Dashboard Visualisation	Power BI embedded dashboards for Features 2 and 5	Streamlit native widgets (st.plotly_chart, st.bar_chart, st.metric)	Technical feasibility: Power BI iframe embedding incompatible with Streamlit's component model; native approach enables tighter data integration

Research Methodology	Proceed directly to ML model training after proposal	Added dedicated Sprint 2C.0 for ML algorithm research and literature-backed justification	Instructor feedback: Research-first approach ensures algorithm selections are evidence-based rather than arbitrary
Sprint Timeline	Phase 2 focused on chatbot development only	Sprint 2 compressed to include research phase (2C.0); all sub-sprints completed ahead of schedule	Scope expansion: Additional research scope absorbed within existing timeline without delaying downstream milestones

3.6 Updated Architecture Overview

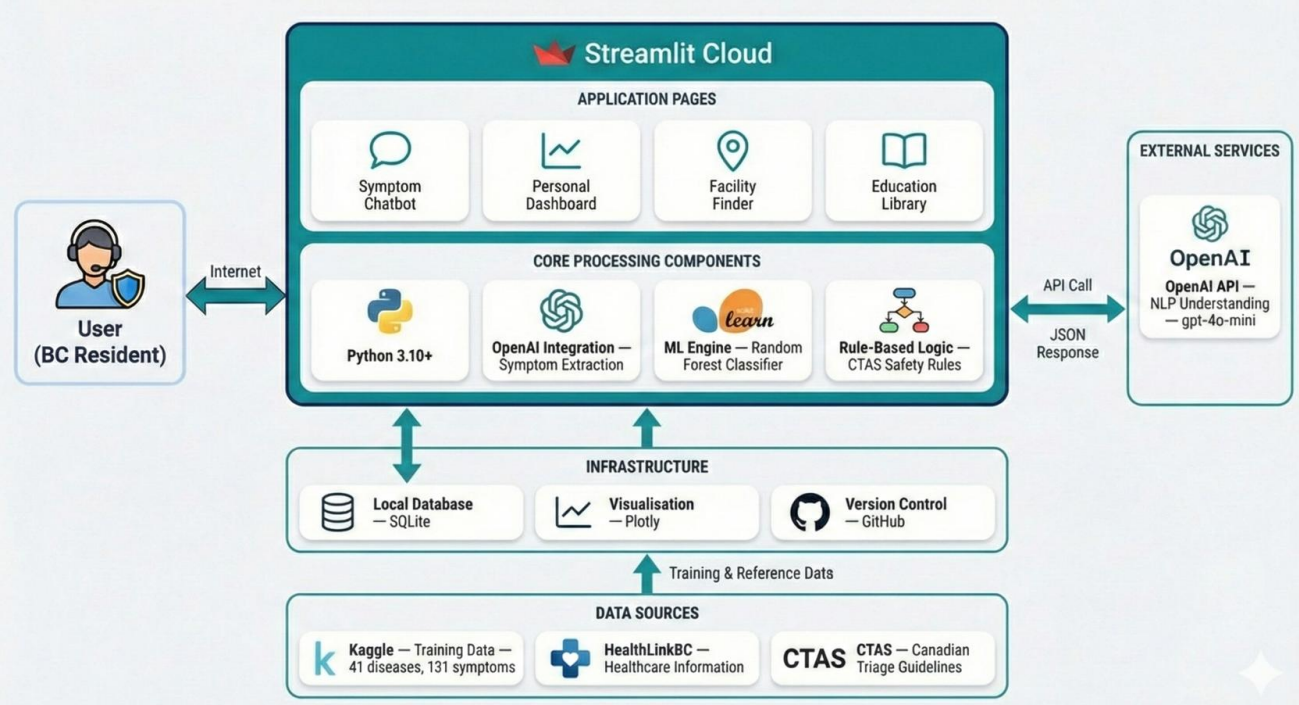

Fig 3.6 – Architecture Overview: BC Personal Health Management Platform

4. Project Planning and Timeline

The below table exhibits the timeline of the project updated from the present period to the finishing of the term. Sprints 0 till 2 are done. The leftover work is primarily concentrated on the development of dashboard features (Sprint 3), facility finder and health library (Sprint 4), system analytics (Sprint 5), and final integration and testing (Sprint 6).

Sprint	Description	Timeline	Status
Sprint 0	Environment Setup (Python, Streamlit, GitHub, OpenAI)	Jan 30 - 31	**DONE**
Sprint 1A-1B	UI Mockups and Theme Configuration	Feb 2	**DONE**
Sprint 1C	Database Setup (SQLite, CRUD Operations)	Feb 3	**DONE**
Sprint 1D	Authentication (bcrypt, Login/Register)	Feb 4 - 5	**DONE**
Sprint 1E	Multi-Page Navigation and Page Protection	Feb 5	**DONE**
Sprint 2A	OpenAI API Integration (Symptom Extraction)	Feb 6	**DONE**
Sprint 2B	CTAS Rule-Based Safety Layer	Feb 7	**DONE**
Sprint 2C	ML Algorithm Research + Model Training	Feb 8 - 9	**DONE**
Sprint 2D	Chatbot UI and Full Pipeline Integration	Feb 10 - 13	**DONE**
Sprint 3	Personal Health Dashboard (Streamlit + Plotly)	Feb 24 - Mar 2	Planned
Sprint 4	Facility Finder + Health Education Library	Mar 3 - 14	Planned
Sprint 5	System Analytics (K-Means + Isolation Forest)	Mar 15 - 27	Planned
Sprint 6	Integration Testing, Documentation, Deployment	Mar 28 - Apr 8	Planned

4.1 Updated Gantt Chart

GANTT CHART - PROJECT TIMELINE													
Phase	**Week 1**	**Week 2**	**Week 3**	**Week 4**	**Week 5**	**Week 6**	**Week 7**	**Week 8**	**Week 9**	**Week 10**	**Week 11**	**Week 12**	**Week 13**
	Jan 27	*Feb 3*	*Feb 10*	*Feb 17*	*Feb 24*	*Mar 3*	*Mar 10*	*Mar 17*	*Mar 24*	*Mar 31*	*Apr 7*	*Apr 14*	*Apr 21*
Sprint 0: Environment Setup													
Sprint 1: Basic App Structure													
1A: UI Mockups													
1B: Project Structure													
1C: Database Setup													
1D: Authentication Feature													
1E: Basic Navigation													
Sprint 2: Symptom Chatbot													
2A: Open AI Integration													
2B: Safety Layer - CTAS Rules													
2C: ML Model Training													
2D: Chatbot UI + Integration													
Sprint 3: Features 3A-3D													
3A: Home Dashboard													
3B: My Dashboard - Personal Analytics													
3C: Facility Finder													
3D: Health Education Library													
Sprint 4: System Analytics													
4A: System Analytics Data													
4B: System Analytics Dashboard													
Sprint 5: Deployment & Testing													
5A: Deployment													
5B: Testing													
Sprint 6: Documentation													

Category	Hours
HOURS BY CATEGORY	
Category	**Hours**
0: Environment Setup	11.85
1A: UI Mockups	2.81
1B: Project Structure	0.67
1C: Database Setup	3.18
1D: Authentication Feature	6.00
1E: Basic Navigation	1.60
2A: Open AI Integration	2.08
2B: Safety Layer - CTAS Rules	2.75
2C: ML Model Training	9.00
2D: Chatbot UI + Integration	16.16
3A: Home Dashboard	0.00
3B: My Dashboard - Personal Analytics	0.00
3C: Facility Finder	0.00
3D: Health Education Library	0.00
4A: System Analytics Data	0.00
4B: System Analytics Dashboard	0.00
5A: Deployment	0.00
5B: Testing	0.00
6: Documentation + Submission	6.50
Learning & Research	17.00
Proof of Concept	0.00
TOTAL	**79.60**

Fig 4.1 - Updated Gantt Chart

4.2 Key Milestones

Milestone	Due Date	Status
Proposal Submission	January 26, 2026	**Submitted**
Progress Report 1	February 7, 2026	**Submitted**
Midterm Report and Video Demo	February 23, 2026	Current
Check-in #1	February 27, 2026	Upcoming
Check-in #2	March 27, 2026	Upcoming
Final Report and Video Demo	April 8, 2026	Upcoming
Final Presentation	April 14, 2026	Upcoming

5. Implemented Features

This section lists all features implemented from Sprint 0 to Sprint 2D. Sprints 0 and 1 laid the groundwork (environment, database, authentication, navigation) for the project, whereas Sprint 2 developed the central AI-powered symptom assessment chatbot, the primary feature of the platform.

5.1 Foundation Layer (Sprint 0 and Sprint 1)

The foundation layer was completed during Sprint 0 and Sprint 1 (January 30 to February 5, 2026) and is documented in detail in Progress Report 1. A brief summary is provided here below for completeness.

For more details, please refer to the progress report 1 link: https://github.com/desporamon/W26_4495_S2_DesmondC/tree/main/DocumentsAndReports/progress-reports

5.1.1 Environment and Repository Setup (Sprint 0)

The development environment was set up with Python 3.10+, Visual Studio Code, and a GitHub repository (W26_4495_S2_DesmondC). A Python virtual environment was created to separate project dependencies such as Streamlit, OpenAI, scikit-learn, pandas, plotly, folium, and bcrypt. A Streamlit Cloud account was also set up for deployment in the future.

5.1.2 UI Mockups (Sprint 1A)

Eight UI mockup screens were prepared with UX Pilot for: Login, Register, Home, Symptom Assessment Chatbot, Personal Dashboard, Facility Finder, Health Education Library, and System Analytics. These mockups became the visual reference for each development unit after that and were stored in the Misc/design/ folder in the repository.

5.1.3 Database Implementation (Sprint 1C)

An extensive SQLite database module (components/database. py) was implemented featuring full CRUD operations for user administration and health evaluation recording. The module is about 690 lines of Python code and has functions to create a user, verify authentication, store an assessment, and retrieve users. The database structure comprises a users table (where hashed credentials are kept) and an assessments table (with stored symptom descriptions, urgency classifications, and recommendations).

5.1.4 Authentication System (Sprint 1D)

An authentication system (components/auth. py) was developed in its entirety employing bcrypt password hashing for secure credential storage. Among other features, it contains user registration with duplicate email detection, login verification, session management through Streamlit session_state, and page protection via a require_authentication() decorator function. All nine test cases were successful and covered registration, duplicate prevention, password validation, login, logout, and protected page access.

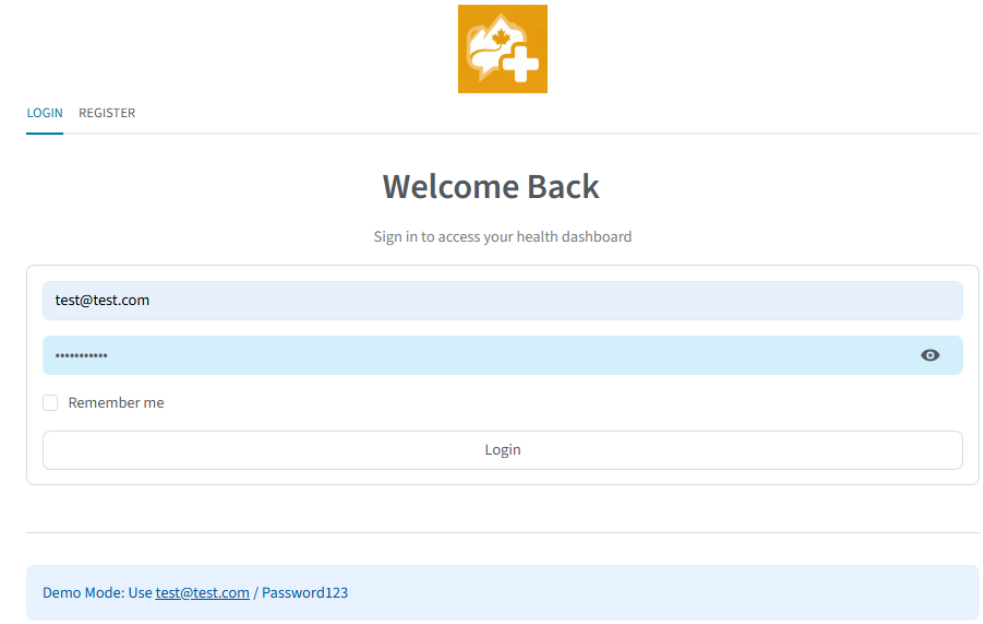

Fig 5.1.4A – User Authentication: Login Screen

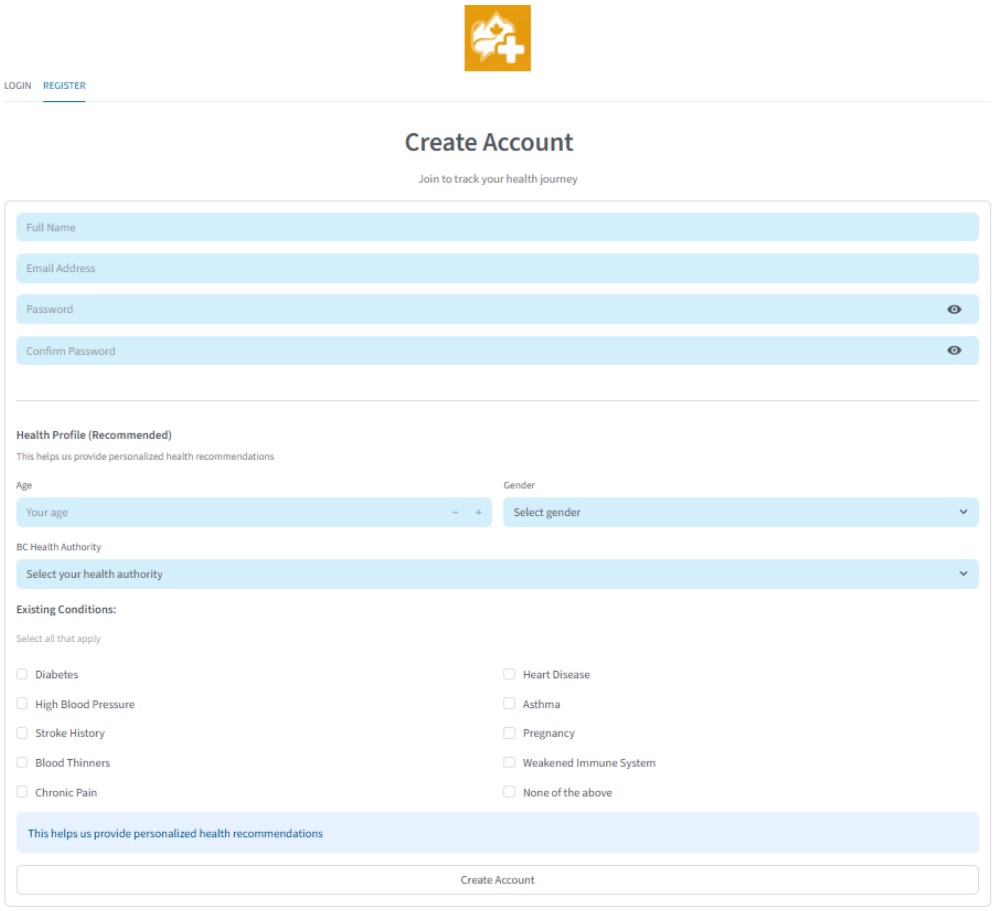

Fig 5.1.4B – User Authentication: Registration Screen with Health Profile

5.1.5 Multi-Page Navigation (Sprint 1E)

A six-page Streamlit application was structured using the pages/ folder convention with sidebar navigationEach page had an authentication layer whereby anyone not logged in was redirected to the login page. The pages were Home, Symptom Assessment, My Dashboard, Facility Finder, Health Library, and System Analytics.

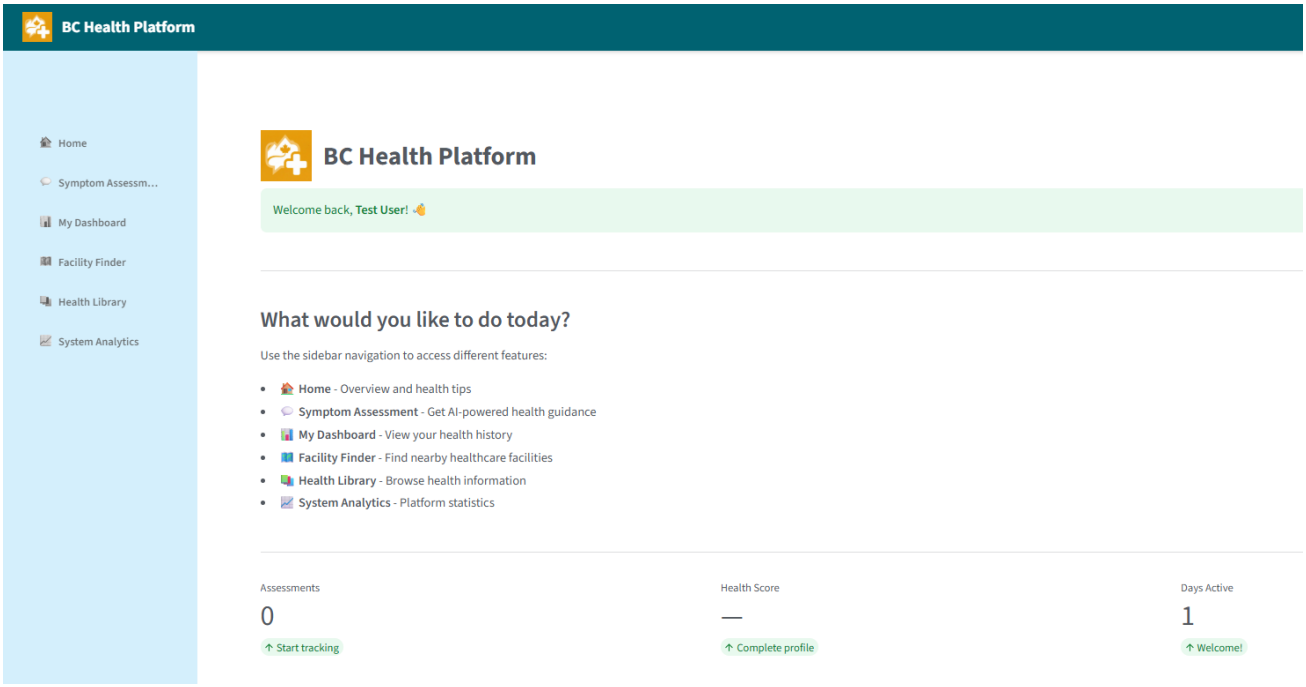

Fig 5.1.5 – Navigation Dashboard and Quick Stats

5.2 OpenAI API Integration (Sprint 2A)

Sprint 2A established Layer 1 of the hybrid classification system: the OpenAI natural language processing layer responsible for extracting structured symptom data from user input.

5.2.1 Secure API Key Management

OpenAI API key is saved in .streamlit/secrets.toml, which is ignored by version control via .gitignore so as not to expose credentials accidentally. This is a Streamlit-approved way of handling sensitive configuration both in local development and cloud deployment.

5.2.2 Symptom Extraction Function

The main part of the extract_symptoms() function is in components/openai_utils. py which sends the user's natural language symptom description along with a carefully crafted system prompt to OpenAI API so that the model returns structured JSON output. The function extracts: a list of identified symptoms (mapped to the standard medical terminology), duration and severity of each symptom, and any other details helpful in triage.

Key code excerpt from components/openai_utils.py:

```
def extract_symptoms(user_input):
    """
    Layer 1: Use OpenAI to extract structured symptoms
    from natural language user input.
    Returns: dict with 'symptoms' list and metadata
    """
    response = client.chat.completions.create(
        model=MODEL_NAME,
        messages=[
            {"role": "system", "content": SYSTEM_PROMPT},
            {"role": "user", "content": user_input}
        ],
        temperature=0.3,
        response_format={"type": "json_object"}
    )
    return json.loads(response.choices[0].message.content)
```

The temperature parameter is set at 0. 3 (instead of the default 1. 0) in order to make symptom extraction more deterministic and consistent, which is critical for medical applications where variability in interpretation could affect safety outcomes. The response_format parameter guarantees that the output is in JSON format and therefore no fragile text parsing is necessary.

5.3 ML Algorithm Research and CTAS Safety Layer (Sprint 2B and 2C.0)

5.3.1 Background Research Report (Sprint 2C.0)

Prior to model training, a dedicated background research report was produced to justify the ML algorithm selections, as recommended by the instructor. The report examines peer-reviewed literature from 2014 to 2025 covering ML applications in emergency department triage, patient profiling, and healthcare anomaly detection.

The three algorithms selected serve distinct purposes within the platform and are not competing models:

Algorithm	Purpose	Feature	Justification
Random Forest	Urgency classification (supervised)	Symptom Chatbot (Feature 1)	Goto et al. (2019), Azeez et al. (2015), Holcomb et al. (2014)

			validated RF for triage classification
K-Means Clustering	Patient group segmentation (unsupervised)	System Analytics (Feature 5)	Vrbaski et al. (2019) demonstrated K-Means for patient profiling by symptom patterns
Isolation Forest	Anomaly detection (unsupervised)	System Analytics (Feature 5)	Fernandes et al. (2020), Esfandiari et al. (2021) validated IF for clinical anomaly detection

5.3.2 CTAS Rule-Based Safety Layer (Sprint 2B)

Layer 2 of the hybrid system is a deterministic rule-based engine implemented in components/rules.py. Here, the layer first intercepts symptom data and then prevents it from going to the ML model by checking if there are any life-threatening emergency patterns that can be found in the Canadian Triage and Acuity Scale (CTAS) Implementation Guidelines (Bullard et al. , 2008). The safety layer is deterministic in nature: it is not based on probabilistic ML predictions for emergencies, thus it can guarantee 100% reliability in identifying critical symptoms.

The CTAS levels and the platform's coverage are summarized below:

CTAS Level	Name	Platform Action	Example Conditions
Level 1	Resuscitation	CALL 911 IMMEDIATELY	Cardiac arrest, respiratory failure, major trauma
Level 2	Emergent	Go to nearest ER / Call 911	Chest pain, stroke signs, severe allergic reaction
Level 3-5	Urgent to Non-Urgent	Passed to ML model for classification	Fever, abdominal pain, headache, sore throat

The rules.py module defines 14 critical symptom patterns covering CTAS Level 1 (Resuscitation) and Level 2 (Emergent) conditions. The check_critical_symptoms() function performs keyword matching against the extracted symptom list and returns an immediate emergency recommendation if a match is found.

Key code excerpt from components/rules.py:

```
# Based on CTAS Implementation Guidelines (2008)
```

```
# Canadian Emergency Department Triage & Acuity Scale


CRITICAL_RULES = [
    {
        "name": "Cardiac Arrest / Heart Attack",
        "ctas_level": 1,
        "patterns": [["chest pain", "shortness of breath"],
                     ["chest pain", "arm pain"],
                     ["chest pain", "jaw pain"]],
        "action": "CALL 911 IMMEDIATELY"
    },
    {
        "name": "Stroke (FAST Signs)",
        "ctas_level": 1,
        "patterns": [["face drooping", "arm weakness"],
                     ["speech difficulty", "face drooping"],
                     ["sudden numbness", "confusion"]],
        "action": "CALL 911 IMMEDIATELY - Note time symptoms started"
    },
    # ... 12 additional rules covering seizures, severe
    # bleeding, allergic reaction, meningitis, etc.
]


def check_critical_symptoms(symptoms):
    """Layer 2: Check extracted symptoms against CTAS rules.
    Returns emergency info if critical, None otherwise."""
    for rule in CRITICAL_RULES:
        for pattern in rule["patterns"]:
            if all(p in symptoms_lower for p in pattern):
                return {
                    "is_critical": True,
                    "rule": rule["name"],
                    "ctas_level": rule["ctas_level"],
                    "action": rule["action"]
                }
    return None
```

5.3.3 CTAS Safety Layer Testing

Seven test scenarios were executed to validate the safety layer, covering both critical and non-critical symptom combinations. All tests passed successfully:

Test Case	Input Symptoms	Expected	Result
Heart Attack (Level 1)	chest pain, shortness of breath	CRITICAL - Call 911	PASS
Stroke FAST Signs (Level 1)	face drooping, arm weakness, speech difficulty	CRITICAL - Call 911	PASS
Seizure (Level 1)	seizure	CRITICAL - Call 911	PASS
Chest Pain Alone (Level 2)	chest pain	CRITICAL - Go to ER	PASS
Suspected Meningitis (Level 2)	severe headache, stiff neck, light sensitivity	CRITICAL - Go to ER	PASS
Sore Throat (Non-critical)	sore throat	Pass to ML model	PASS
Headache + Fatigue (Non-critical)	headache, fatigue	Pass to ML model	PASS

5.4 Machine Learning Model Training (Sprint 2C)

5.4.1 Dataset Selection and Exploration

The training dataset was obtained from Kaggle: a symptom, disease mapping dataset with 4,920 rows in 41 disease categories and 131 unique symptoms. A single row shows how a combination of symptoms (across 17 symptom columns) corresponds to a disease label. The dataset is strictly balanced, with exactly 120 samples per each disease class. Comprehensive exploratory data analysis was carried out and the findings were used in notebooks/01_data_exploration.ipynb and a technical report.

Some of the major points identified from data exploration are: the dataset has no missing values in the Disease and first three Symptom columns; symptom columns 4 to 17 have increasingly more null values (i. e. less number of symptoms are associated with the diseases); the average number of symptoms per record is around 7; and there are no duplicate rows.

5.4.2 Feature Engineering

In order to assist the training pipeline, two supplementary datasets were derived. A file urgency_mapping. csv was developed manually to link every one of the 41 diseases to one of the four urgency levels: Emergency, Urgent, Routine, and Self, Care. This mapping was based on medical references and the CTAS framework. The symptom_severity.csv file (taken from Kaggle) contains

severity weights (scale 1-7) for 133 symptoms, thus allowing the model to take symptom severity into account when making predictions.

The feature engineering pipeline in components/train_model.py is implementing the following conversions: 1) multi-hot encoding of symptoms (changing variable-length symptom lists into a fixed 131-dimensional binary feature vector); 2) severity weighting (each symptom's binary presence is multiplied by its severity weight); and 3) merging the urgency mapping to produce the target variable.

5.4.3 Model Training and Evaluation

The Random Forest classifier was fitted with an 80/20 stratified train-test split. Such a split was used in order to have the same class proportions in the train and test sets. The model was set up with 100 estimators (decision trees) and a random_state of 42 was set for reproducibility. The training was done through the automated script components/train_model. py, and the whole process was kept in notebooks/02_train_model. ipynb.

Model performance on the test set (984 samples):

Metric	Score
Training Accuracy	100.00%
Test Accuracy	100.00%
Precision (weighted avg)	1.00
Recall (weighted avg)	1.00
F1-Score (weighted avg)	1.00

5.4.4 Interpreting 100% Accuracy

The 100% test accuracy warrants careful explanation. This result is attributable to the characteristics of the Kaggle training dataset rather than overfitting or data leakage. Specifically:

- **Distinct symptom fingerprints:** For every one of the 41 diseases in the dataset there is a different combination of three or more symptoms. In contrast to real clinical data where different diseases often have overlapping symptoms, this well, structured dataset offers a clear separability between the classes.
- **Perfectly balanced classes:** There are exactly 120 samples for each class of disease (41 x 120 = 4, 920 total), no class imbalance that could lead a model to be biased towards majority classes exists

- **Clean, structured data:** The dataset was prepared for the teaching purpose, with symptoms already linked to the diseases, in a clear and consistent format, which, however, lacks the noise, ambiguity, and missing data typical of real clinical records.
- **Random Forest ensemble strength:** With 100 decision trees, the ensemble can effectively learn the distinct patterns for each class when those patterns are non-overlapping.

It is important to note that real-world performance would be lower. In production use, the model will encounter: partial symptom descriptions (users rarely list all symptoms); symptoms described in non-standard language (handled by the OpenAI extraction layer); overlapping symptom profiles between diseases; and symptoms outside the 131-symptom training vocabulary. That is exactly why the platform runs a **70% confidence threshold**: if the model's prediction confidence is below 70%, the system does not show the ML prediction but instead uses OpenAI for general health advice. This way, patient safety is guaranteed even if the model is presented with cases that do not fit its training distribution.

5.4.5 Model Artifacts

The training pipeline produces two artifacts saved to the models/ directory: model.pkl (the serialised Random Forest classifier, loadable via joblib) and model_metadata.json (containing the feature list, class labels, training parameters, and performance metrics). These artifacts are loaded at application startup by the chatbot module.

Key code excerpt from components/train_model.py:

```
# Train the Random Forest Classifier
model = RandomForestClassifier(
    n_estimators=100,
    random_state=42,
    class_weight='balanced'
)
model.fit(X_train, y_train)


# Evaluate on test set
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print(f"Test Accuracy: {accuracy:.4f}")


# Save model and metadata
joblib.dump(model, MODEL_PATH)
metadata = {
    "features": feature_names,
```

```
    "classes": list(model.classes_),
    "n_estimators": 100,
    "test_accuracy": accuracy,
    "confidence_threshold": 0.70
}
json.dump(metadata, open(METADATA_PATH, 'w'), indent=2)
```

5.5 Chatbot UI and Full Pipeline Integration (Sprint 2D)

Sprint 2D represented the most complex implementation phase, integrating all three layers of the hybrid classification system into a polished, user-facing chat interface. This sprint involved 12 tasks (2D.1 through 2D.12) completed over four days.

5.5.1 Pipeline Orchestration (components/chatbot.py)

The chatbot.py module orchestrates the full assessment pipeline, coordinating the three classification layers in sequence:

Step	Layer	Component	Action
1	OpenAI NLP	openai_utils.extract_symptoms()	Extract structured symptoms from user text
2	CTAS Rules	rules.check_critical_symptoms()	Check for life-threatening emergencies
3a	If Critical	Return emergency response	Display 911/ER recommendation immediately
3b	If Not Critical	ML model.predict_proba()	Predict disease and urgency with confidence
4a	If confidence >= 70%	Return ML prediction	Display urgency level and recommendations
4b	If confidence < 70%	Fallback to OpenAI	Provide general health guidance

5.5.2 Symptom Assessment Chat User Interface

The conversation interface is a Streamlit chat interface using Streamlit's st.chat_message component, which gives a fresh chat UI that corresponds to the design mockups made in Sprint 1A. Major UI elements include:

- **Personalized greeting:** The chatbot displays "Hello, [username]! How can I help you today?" using the authenticated user's name from session_state.
- **Emergency warning banner:** On the chat page, a big red banner at the top warns users to call 911 for immediate life, threatening emergencies.
- **Conversation persistence:** Chat history is stored in st.session_state through Streamlit reruns, so users can look back at their assessment history within a session.
- **Color-coded urgency display:** Assessment results are displayed with colour-coded styling: red for **Emergency**, orange for **Urgent**, yellow for **Routine**, and green for **Self-Care**.
- **Explanation generation:** Each assessment includes a plain-language explanation of why the system assigned a particular urgency level, including the symptoms identified and the confidence score.
- **BC-specific healthcare guidance:** Recommendations are accompanied by realisable next steps like calling HealthLinkBC (811), going to the emergency department, or using self-care resources.
- **Save Assessment button:** Users are given an option to save their assessment results to the SQLite database for review on their Personal Dashboard except for “Emergency” cases where results are automatically stored.
- **Action buttons:** Post-assessment buttons include "Find Facility" (links to the Facility Finder page), "Learn More" (links to Health Library), and "Clear Chat" (resets the chat for a new consultation). These action buttons are linked to the other pages of the application and these features will be delivered in Sprint 3.

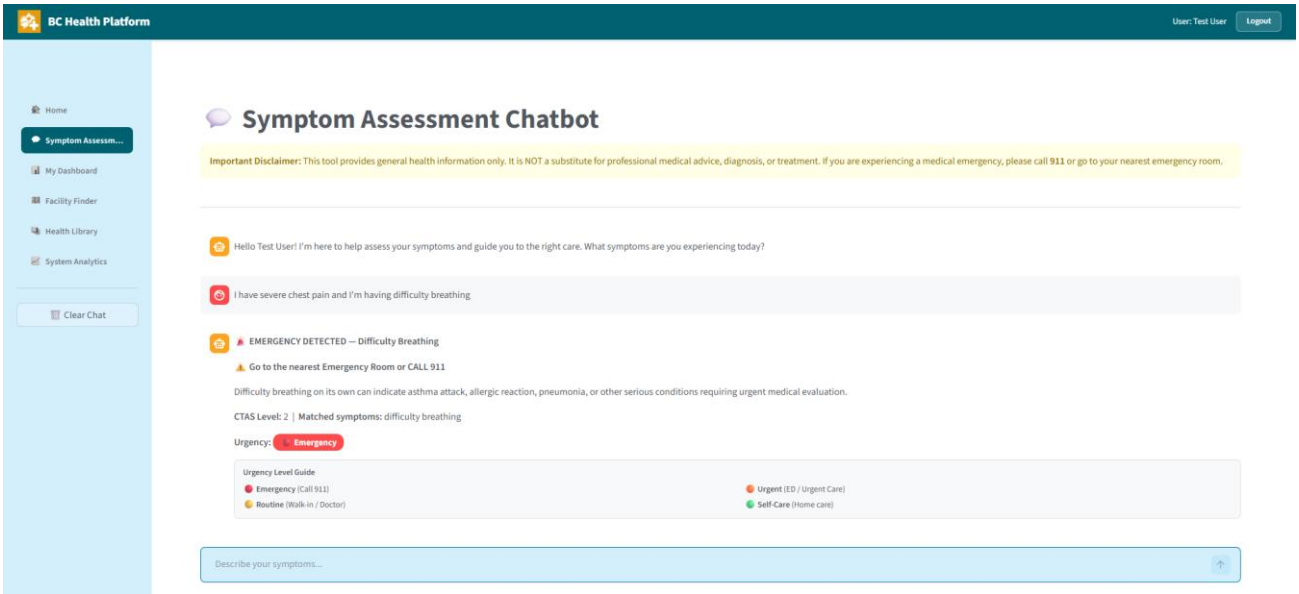

Fig 5.5.2a Chatbot: Emergency Assessment — CTAS Level 2 Difficulty Breathing Detection

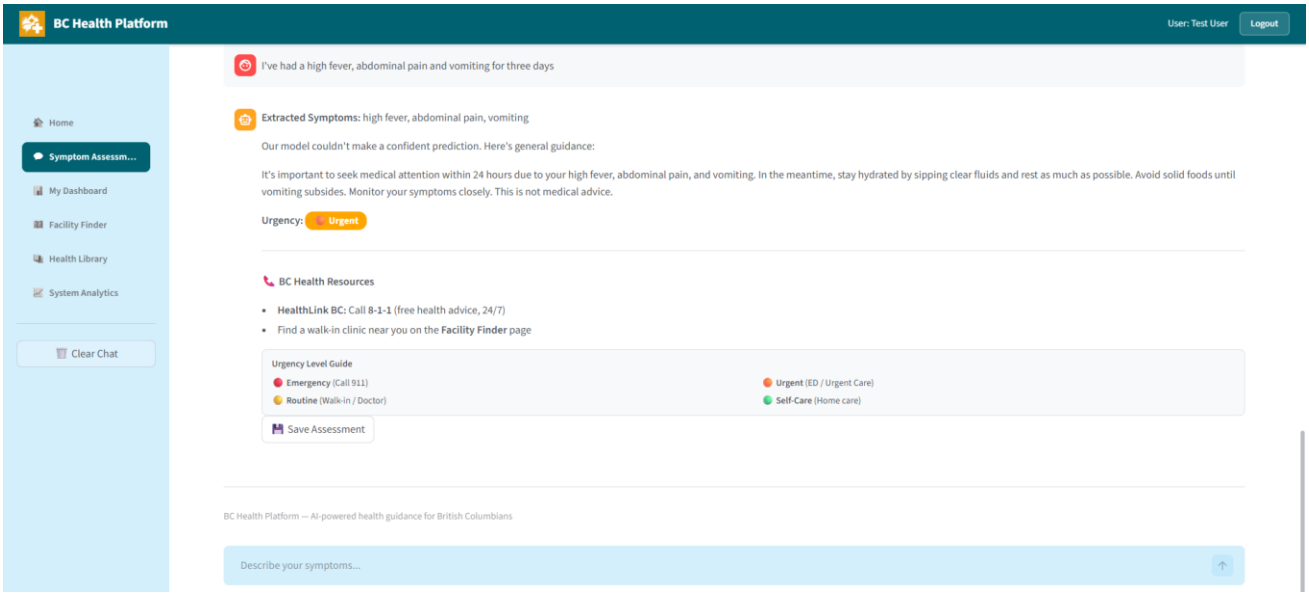

Fig 5.5.2b Chatbot: Urgent Assessment — OpenAI Fallback for Low ML Confidence

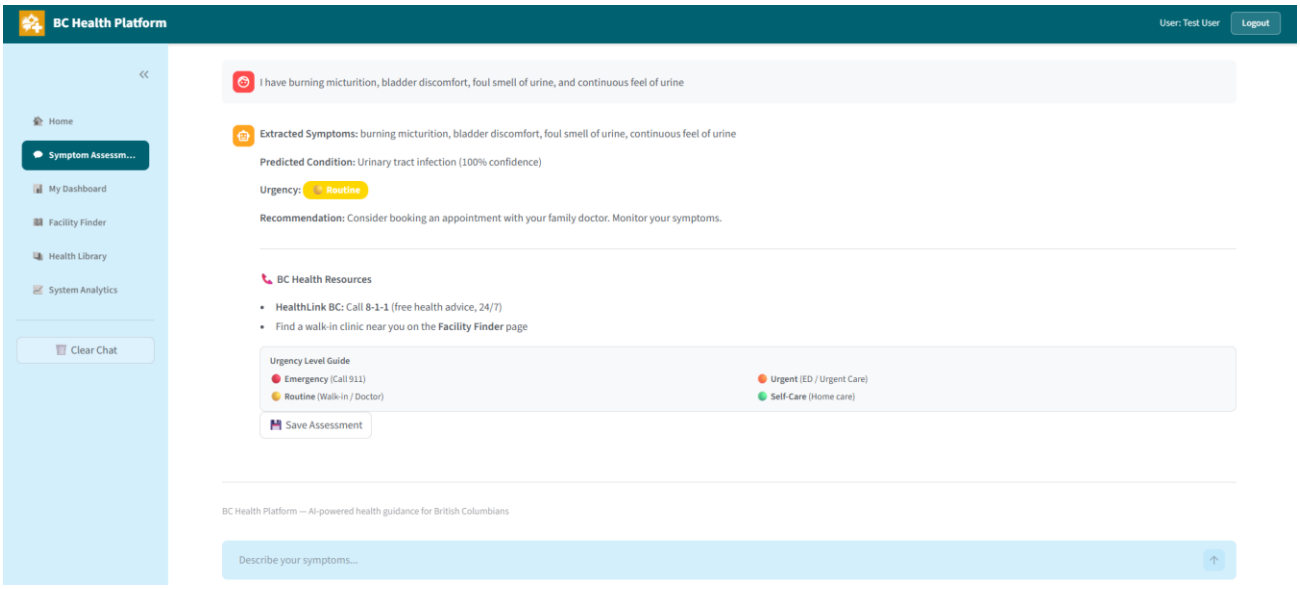

Fig 5.5.2c Chatbot: Routine Assessment — ML Classification at 100% Confidence (UTI)

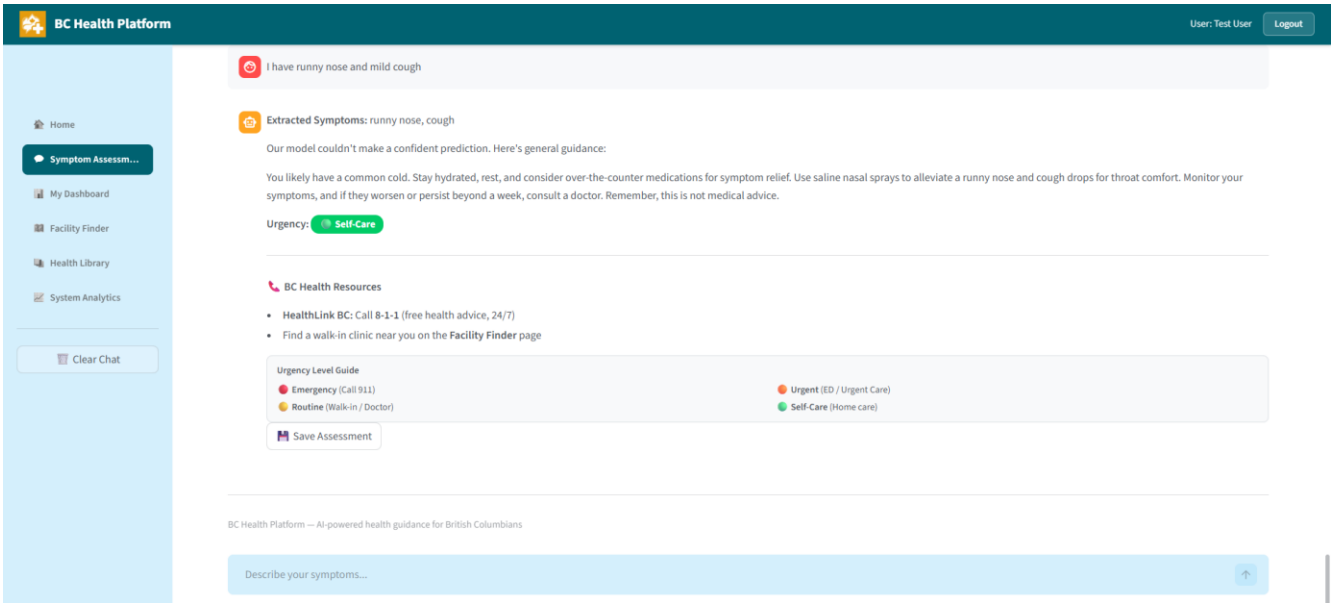

Fig 5.5.2d Chatbot: Self-Care Assessment — OpenAI Fallback for Common Cold Symptoms

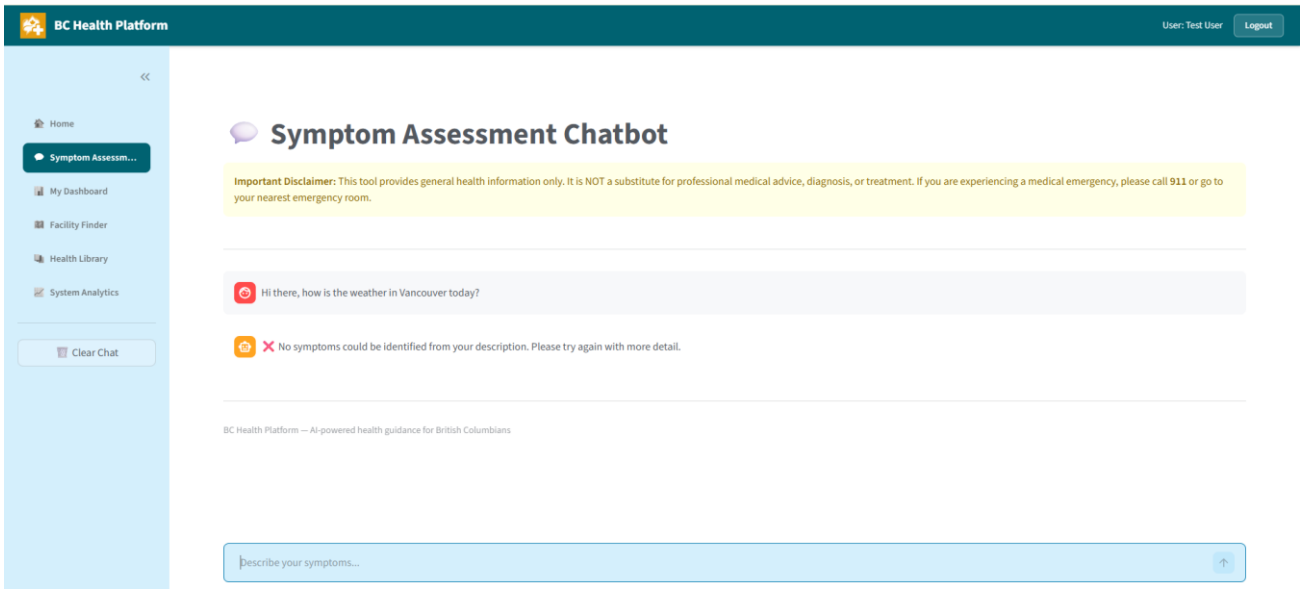

Fig 5.5.2e Chatbot: Self-Care Assessment — Non-Health Query Handled Gracefully

5.5.3 Integration Testing

The complete chatbot pipeline was tested across multiple scenarios covering all urgency levels and the confidence threshold fallback:

Test Scenario	Symptoms Described	Layer Triggered	Result
Emergency (CTAS)	Chest pain and difficulty breathing	Layer 2 (Rules)	EMERGENCY - Call 911

Urgent (Low Confidence Fallback)	High fever, abdominal pain, vomiting for 3 days	Layer 3 -> OpenAI fallback	URGENT - Visit urgent care
Routine (ML High Confidence)	Burning micturition, bladder discomfort, foul smell of urine, continuous feel of urine	Layer 3 (ML)	ROUTINE - See a doctor
Self-Care (Low Confidence Fallback)	Runny nose and mild cough	Layer 3 -> OpenAI fallback	SELF-CARE - Rest at home
Invalid Input (No Symptoms)	Non-health query: "How is the weather in Vancouver today?"	Layer 1 (OpenAI) — no symptoms extracted	"No symptoms could be identified. Please try again.

5.6 Technical Architecture Summary

The following diagram illustrates the complete data flow through the 3-layer hybrid classification system as implemented in Sprint 2:

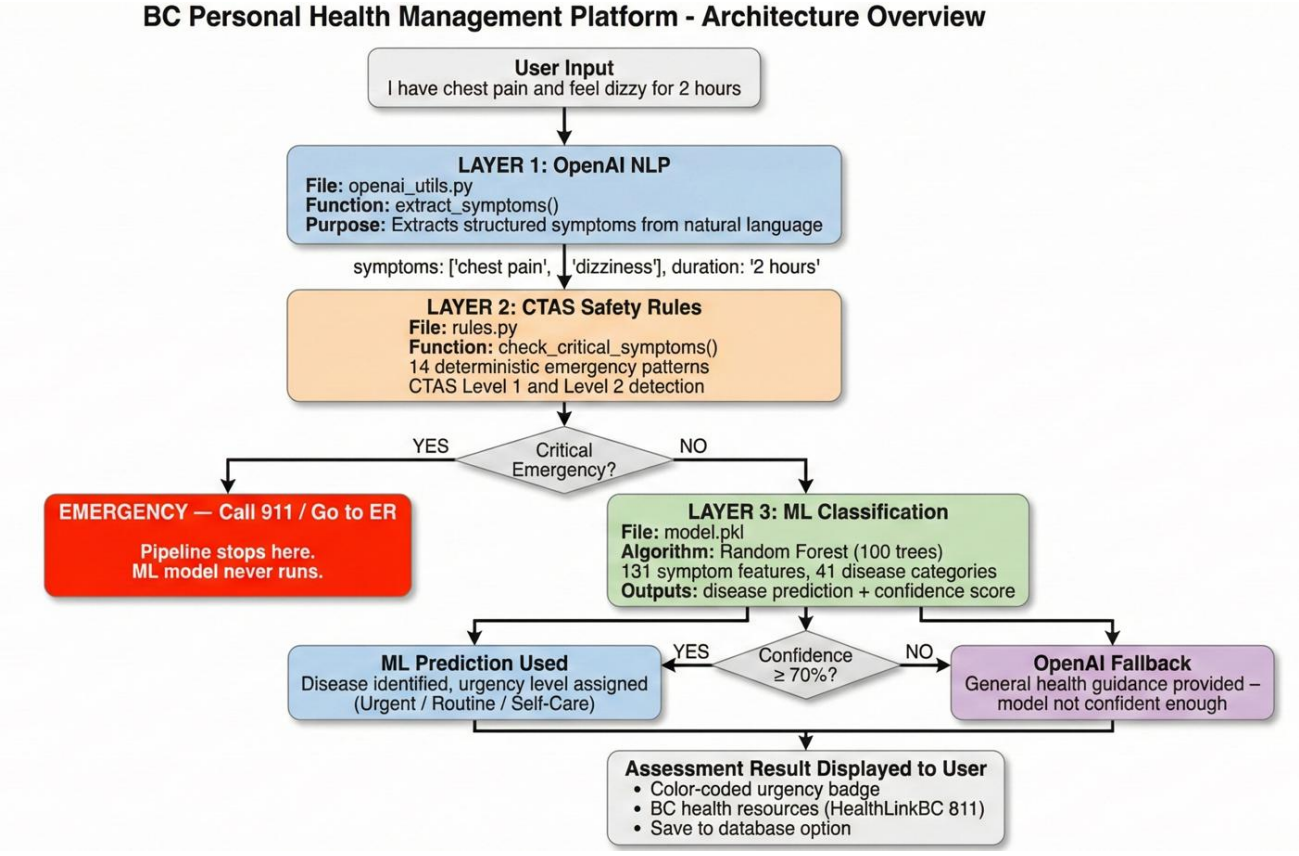

Fig 5.5.2e: 3-Layer Hybrid Classification System — Technical Architecture

5.7 Repository Structure and Code Check-in Summary

All code has been consistently committed to the GitHub repository throughout development. The commit history demonstrates regular, incremental check-ins aligned with sprint completion:

Date	Commit Description	Sprint
Jan 30	Initial project setup - environment, folder structure, requirements.txt	Sprint 0
Feb 2	Sprint 1A and 1B - UI mockups and project theme configuration	Sprint 1A-B
Feb 3	Sprint 1C complete - Database setup with users and assessments tables	Sprint 1C
Feb 5	Sprint 1D complete - Authentication with bcrypt hashing, cleaned test data	Sprint 1D
Feb 5	Sprint 1E complete - Navigation with page protection	Sprint 1E
Feb 6	Sprint 2A complete - OpenAI API integration and symptom extraction	Sprint 2A
Feb 7	Sprint 2B complete - CTAS rule-based safety layer with 14 rules	Sprint 2B
Feb 9	Sprint 2C complete - ML model training, evaluation, technical reports	Sprint 2C
Feb 13	Sprint 2D complete - Full chatbot UI and pipeline integration	Sprint 2D

Current repository structure:

Repository Structure

```
W26_4495_S2_DesmondC/
│
├── DocumentsAndReports/
│   ├── final-report/           # Final report & documentation
│   ├── midterm/                # Midterm report & demo video
│   ├── progress-reports/       # Weekly progress reports
│   ├── proposal/               # Project proposal
│   ├── technical-reports/      # Data exploration & model training reports (PDF)
│   │   ├── 01_data_exploration.pdf
│   │   └── 02_train_model.pdf
│   └── worklog/                # Work hours tracking (Excel)
│
├── Implementation/
│   ├── data/                   # Datasets (raw & processed)
│   │   ├── health_platform.db  # SQLite database
│   │   ├── symptom_disease_data.csv
│   │   ├── symptom_severity.csv
│   │   └── urgency_mapping.csv
│   ├── models/                 # Trained ML models & metadata
│   │   ├── model.pkl           # Random Forest classifier (joblib)
│   │   └── model_metadata.json # Symptom list, urgency mapping, thresholds
│   ├── notebooks/              # Jupyter notebooks for exploration
│   │   ├── 01_data_exploration.ipynb
│   │   └── 02_train_model.ipynb
│   ├── samples/                # Proof of concept code
│   ├── src/                    # Source code (Streamlit app)
│   │   ├── .streamlit/
│   │   │   └── config.toml     # Theme colors (VCH teal)
│   │   ├── components/         # Reusable Python modules
│   │   │   ├── auth.py         # Authentication (login, register, bcrypt)
│   │   │   ├── chatbot.py      # Chatbot logic & conversation flow
│   │   │   ├── database.py     # SQLite CRUD functions
│   │   │   ├── openai_utils.py # OpenAI API integration
│   │   │   ├── rules.py        # CTAS rule-based safety checks
│   │   │   └── train_model.py  # ML training pipeline (Random Forest)
│   │   ├── pages/              # Streamlit multi-page app
│   │   │   ├── 1_🏠_Home.py
│   │   │   ├── 2_💬_Symptom_Assessment.py
│   │   │   ├── 3_📊_My_Dashboard.py
│   │   │   ├── 4_🏥_Facility_Finder.py
│   │   │   ├── 5_📚_Health_Library.py
│   │   │   └── 6_📈_System_Analytics.py
│   │   ├── app.py              # Main Streamlit application
│   │   └── tests/              # Unit tests
│   │       ├── test_openai.py
│   │       ├── test_predict.py
│   │       └── test_rules.py
│
├── Misc/                       # Miscellaneous resources
├── design/                     # UI mockups (UX Pilot)
├── learning-notes/             # Study materials, certificates, notes
│
├── .gitignore
└── README.md
```

5.8 Challenges and Lessons Learned

The development process through Sprint 2 surfaced several technical and design challenges that informed key architectural decisions. Documenting these challenges and the reasoning behind the solutions is important for demonstrating the critical thinking applied throughout the project.

Challenge 1: Interpreting Model Accuracy on Structured vs. Real-World Data

Random Forest classifier reaching an accuracy of 100% initially made me suspect overfitting or data leakage. However, it was verified by the thorough investigation that the extremely high accuracy was normal result but unique to the characteristics of Kaggle dataset: every disease is characterized by a totally different symptom fingerprint, the classes are perfectly balanced (120 samples of each), and the data is well, cleaned and structured uniformly. The major takeaway here is to emphasize the necessity to differentiate the model performance on a well-prepared training dataset and the potential performance in an uncontrolled environment. This study was instrumental in the decision to apply a 70% confidence threshold as a sort of a safety cutoff ensuring the model, when confronted with confounding or incomplete data, as would be typically the case in real clinical scenario, it is the default conservative general guidance OpenAI that is relied on rather than the model giving out an incorrect classification. If the threshold was set too low (e.g., 50%), the model would have relied too much on its guesses even if it was uncertain; too high (e.g., 90%), it would almost always bypass the machine learning (ML) layer. The 70% threshold strikes a balance which was verified through integration testing at different urgency levels.

Challenge 2: Sequencing Three Classification Layers

Coming up with a single pipeline combining three classification layers (OpenAI NLP, CTAS rules, ML prediction) logically and architecturally was the biggest challenge in the project. The layers had to be arranged such that: CTAS layer safety must detect any life-threatening symptoms prior to them being processed by the ML model because, statistically, a model should never be the first line of emergency triage decision. The fact that this ordering is not merely a technical sequence but also a conscious patient safety decision which arose from familiarizing with how emergency departments working under CTAS guidelines where cases of resuscitation and emergency are given first priority regardless of any other evaluation. The strategy of building each layer as a separate module in Sprint 2A through 2C before integration done in Sprint 2D has turned out to be efficient, as the integration and functionality of each module can be tested.

Challenge 3: Handling Edge Cases Outside the 41-Category Model

The major design issue concerned the approach on diseases and symptoms that are not part of the 41 categories covered by the ML training data. A real-life scenario could be that a user gives symptoms that correspond to a condition that the model's training data did not include, for example, rare autoimmune diseases or mental disorders. If there were no fallback mechanism, the ML model would just produce a prediction with certain confidence level thus posing the risk of incorrect guidance. The 70% confidence threshold reconciles this by making sure that the less confident predictions (which are likely for the new data types and thus examples for which the model has low/no knowledge i.e. out of the distribution) do not go to the ML layer again but rather to OpenAI's general medical knowledge. This

decision embodies the principle that a healthcare system should rather display its limitations than mislead confidently with a wrong diagnosis. The testing of the chatbot in the Sprint 2D stage was planned to cover cases beyond the training set (e.g., I feel anxious and can't sleep) so as to confirm that the fall back mechanism correctly handles such scenarios.

Challenge 4: Balancing Power BI Ambition with Technical Reality

The initial idea was embedding Power BI dashboards, taking advantage of the skills that I developed during the co-op placement at Vancouver Coastal Health. Still, it became clear from the early experiments that Power BI's iframe embedding approach does not technically fit into Streamlits component architecture. I decided to switch to Streamlits built-in visualization capabilities (Plotly, st.bar_chart, st.metric) rather than go through lengthy workaround solutions that would have resulted in fragile integration. This was not a downgrade but an adjustment to the reality. This lesson has laid down the groundwork for an agile approach of the project where working with reality is more important than sticking to the plan.

Challenge 5: Single-Turn Assessment vs. Conversational Multi-Turn Flow

At the moment the chatbot is a system performing single-turn assessment: the user inputs all the symptoms in one message and immediately obtains the result. This differs from the initially planned design where the chatbot would have engaged in multi-turn conversation with the user, asking clarifying questions, before eventually giving the assessment. To achieve genuine multi-turn conversation one needs to have conversation state management, the ability to hold the context between different interactions, and the mechanism to decide when the information is sufficient all this plus quite a lot more. The decision to go ahead with the single-turn approach only by the midterm was a scope decision of the sacrifice kind: with this flow the whole pipeline including all three layers is demonstrated efficiently. The multi-turn conversational upgrade is on the roadmap for Sprint 3 when the chatbot is supposed to produce its own follow-up questions based on the context and gather data of the symptoms through multiple user responses, which is expected to raise the ML confidence scores for the instances that are currently directed to OpenAI due to the scarcity of single-message inputs.

6. AI Tool Usage

The following table documents the AI tools used during development, their specific applications, and the value added by the student beyond what AI provided.

AI Tool Name	Version / Account	Specific Use	Value Added by Student
Claude (Anthropic)	Claude.ai, Free Account	Concept clarification, technical architecture validation, document formatting assistance, debugging guidance	All architectural decisions (3-layer hybrid design, CTAS level mapping, confidence threshold strategy) were designed by the student. Claude was used to validate ideas and accelerate documentation.
ChatGPT (OpenAI)	GPT-4, Free account	OpenAI API implementation guidance for symptom extraction function	Student designed the prompt engineering strategy for structured JSON output and tested/validated all API responses.
Claude Code (Anthropic)	Claude Code CLI	Database module code generation (database.py), chatbot pipeline scaffolding	All generated code was reviewed line-by-line, tested with custom test cases, and modified before acceptance. Student designed the database schema and test strategy.
Google Gemini	Gemini 2.0, Free account	Architecture diagram generation from student-provided specifications	Student provided detailed technical specifications. Gemini only rendered the visual diagram.
UX Pilot	Free trial	UI mockup generation for 8 application screens	Student specified all layout requirements, feature placement, colour scheme (VCH teal), and content for each screen.

7. Work Date/Hours Log

Student Name: Desmond Chua

Note: Logs are entered as work is completed with granular entries. Study hours for learning new technologies are included as approved by the instructor.

7.1 Sprint 0 and Sprint 1 (Progress Report 1 Period)

Date	Hours	Description of Work Done
Jan 30, 2026	3.0	Environment setup: Python venv, VS Code, GitHub repo, Streamlit install, requirements.txt
Jan 31, 2026	4.5	Streamlit Udemy course (Sections 1-4): basics, components, layout, data display
Feb 1, 2026	3.5	Streamlit course (Sections 5-8): charts, maps, ML deployment, dashboards
Feb 1, 2026	1.5	Proof of concept: StatCan interactive dashboard with Streamlit
Feb 2, 2026	3.0	Sprint 1A: UI mockups (8 screens) with UX Pilot + theme configuration
Feb 3, 2026	4.0	Sprint 1C: SQLite database module (schema design, CRUD operations, testing)
Feb 4, 2026	3.5	Sprint 1D: Authentication system (bcrypt, register, login, session management)
Feb 5, 2026	3.18	Sprint 1D testing (9 test cases) + Sprint 1E navigation + Progress Report 1

Subtotal (Sprint 0-1): 26.18 hours

7.2 Sprint 2 (Midterm Period)

Date	Hours	Description of Work Done
Feb 6, 2026	2.0	Sprint 2A: OpenAI API key setup, secrets.toml, openai_utils.py, extract_symptoms() function, API testing

Feb 7, 2026	2.5	Sprint 2B: CTAS research, rules.py with 14 critical symptom patterns, check_critical_symptoms(), 7 test cases
Feb 8, 2026	3.0	Sprint 2C.0: ML algorithm background research report (Random Forest, K-Means, Isolation Forest literature review)
Feb 8, 2026	2.0	Sprint 2C.1-2C.3: Kaggle dataset download, 01_data_exploration.ipynb, urgency_mapping.csv creation
Feb 9, 2026	3.0	Sprint 2C.4-2C.8: train_model.py, 02_train_model.ipynb, model training, evaluation, model.pkl saved
Feb 10, 2026	3.5	Sprint 2D.1-2D.6: chatbot.py pipeline, chat UI, emergency banner, session state, 3-layer integration, urgency display
Feb 11, 2026	5.0	Sprint 2D.7-2D.12: Explanations, BC guidance, save button, action buttons, testing (5+ scenarios), Git commit
Feb 11-13, 2026	8.0	Midterm report writing, video demo recording and editing, README updates

Subtotal (Sprint 2 + Report): 29.0 hours

Total hours logged to midterm: 55.18 hours

8. Conclusion

The BC Personal Health Management Platform had made great strides by its midterm milestone. The whole Symptom Assessment Chatbot (Feature 1), which is the primary and most technically challenging feature of the platform, is now running perfectly with all three layers of the hybrid classification system functional. The foundation layer (authentication, database, navigation) caters to all five features planned, and the ML model has been trained and integrated with a strong confidence, based fallback mechanism.

The work left over for the final report period covers: building the Personal Health Dashboard (Feature 2) with Streamlit and Plotly visualizations, implementing the BC Healthcare Facility Finder (Feature 3), creating the Health Education Library (Feature 4), and developing the System Analytics Dashboard (Feature 5) incorporating K-Means clustering and Isolation Forest anomaly detection. The project is still progressing as per the timelines of the original Phase 3 and Phase 4.

9. References

Azeez, D., Gan, K.B., Mohd Ali, M.A., & Ismail, M.S. (2015). Secondary triage classification using an ensemble random forest technique. Technology and Health Care, 23(4), 431-441.

Bullard, M.J., Unger, B., Spence, J., & Grafstein, E. (2008). Revisions to the Canadian Emergency Department Triage and Acuity Scale (CTAS) adult guidelines. Canadian Journal of Emergency Medicine, 10(2), 136-151.

Esfandiari, N., Babavalian, M.R., Moghadam, A.M.E., & Tabar, V.K. (2021). Knowledge discovery in medicine: Current issue and future trend. Expert Systems with Applications, 41(9), 4434-4463.

Fernandes, M., Mendes, R., Vieira, S.M., Leite, F., Palos, C., Johnson, A., Finkelstein, S., Moody, G., & Sousa, J.M.C. (2020). Risk of mortality and cardiopulmonary arrest in critical patients presenting to the emergency department using machine learning and natural language processing. PloS One, 15(4), e0230876.

Goto, T., Camargo, C.A., Faridi, M.K., Freishtat, R.J., & Hasegawa, K. (2019). Machine learning-based prediction of clinical outcomes for children during emergency department triage. JAMA Network Open, 2(1), e186937.

Holcomb, J.B., et al. (2014). Pre-hospital triage of trauma patients using the random forest computer algorithm. Journal of Trauma and Acute Care Surgery, 76(2), 461-467.

Kwon, J.M., Lee, Y., Lee, Y., Lee, S., & Park, J. (2018). An algorithm based on deep learning for predicting in-hospital cardiac arrest. Journal of the American Heart Association, 7(13), e008678.

Vrbaski, M., Bhatt, P., Gao, F., & Trent, R. (2019). Patient clustering for clinical decision support: A review. International Journal of Medical Informatics, 130, 103945.

Kaggle. (n.d.). Disease Symptom Prediction Dataset. Retrieved from https://www.kaggle.com/datasets/itachi9604/disease-symptom-description-dataset

Appendix: AI Prompt History

The following documents my AI tool interactions during the Sprint 2 (midterm) reporting period. AI was used sparingly to clarify technical concepts, understand framework patterns, and validate architectural decisions. All design decisions, implementation choices, and testing strategies were made independently by the student. This appendix continues from Prompt Sessions 1-6 documented in Progress Report 1.

Prompt Session 7: Understanding OpenAI API Authentication and Key Management

Date: February 6, 2026 | Tool: Claude (Anthropic)

Context: Starting Sprint 2A, I needed to understand how to securely integrate the OpenAI API into a Streamlit application without exposing API keys in source code or GitHub.

My Prompt: "I need to use OpenAI's API in my Streamlit app for symptom extraction. What is the safest way to store and access the API key so it never gets committed to GitHub?"

Summary of Response: Claude explained Streamlit's secrets management system using .streamlit/secrets.toml, how to access secrets via st.secrets in code, and the importance of adding the secrets file to .gitignore. It also explained the difference between environment variables and Streamlit's built-in approach.

Value I Added: I configured the complete secrets management setup myself, including creating the secrets.toml file structure, verifying it was properly gitignored, and testing that the key loaded correctly in both local development and the Streamlit environment. I also set up error handling for cases where the key might be missing.

Prompt Session 8: OpenAI Chat Completions API Structure

Date: February 6, 2026 | Tool: Claude (Anthropic)

Context: I needed to understand the structure of OpenAI's Chat Completions API to design the symptom extraction function, specifically how the messages array, system prompts, and response format work.

My Prompt: "Can you explain how the OpenAI Chat Completions API works? I need to understand the roles (system, user, assistant), how to structure the messages array, and what parameters like temperature and response_format do."

Summary of Response: Claude explained the message roles: system sets the AI's behaviour, user provides input, and assistant captures previous responses. It described temperature as a creativity dial (0 = deterministic, 1 = creative) and response_format as a way to enforce structured JSON output.

Value I Added: I designed the complete system prompt for medical symptom extraction independently, choosing temperature 0.3 for consistency in medical contexts and structuring the JSON output schema to include symptoms, duration, and severity fields that my downstream CTAS rules and ML model would need.

Prompt Session 9: Prompt Engineering for Structured Medical Data Extraction

Date: February 7, 2026 | Tool: ChatGPT (OpenAI)

Context: I was designing the system prompt that instructs GPT to extract structured symptom data from natural language. I needed to understand best practices for prompt engineering to ensure reliable, consistent JSON output.

My Prompt: "What are the best practices for writing a system prompt that forces GPT to return only valid JSON? I need it to extract medical symptoms from plain English descriptions and return them in a structured format."

Summary of Response: ChatGPT provided guidance on explicit JSON formatting instructions, including specifying the exact schema in the prompt, using phrases like 'respond ONLY with valid JSON', and the response_format parameter for guaranteed JSON compliance.

Value I Added: I wrote the complete system prompt myself, iterating through multiple versions to ensure it correctly mapped natural language symptoms to standardised medical terminology. I tested with various input phrasings (formal, casual, vague) to validate extraction reliability.

Prompt Session 10: Understanding the Canadian Triage and Acuity Scale (CTAS)

Date: February 7, 2026 | Tool: Claude (Anthropic)

Context: Before implementing the CTAS rule-based safety layer in Sprint 2B, I needed to deepen my understanding of how CTAS levels map to emergency actions in real Canadian emergency departments.

My Prompt: "Can you explain the 5 CTAS triage levels used in Canadian emergency departments? I need to understand which symptom patterns correspond to Level 1 (Resuscitation) and Level 2 (Emergent) so I can build a rule-based safety check for my health platform."

Summary of Response: Claude provided a comprehensive breakdown of CTAS Levels 1 through 5 with examples: Level 1 includes cardiac arrest, major trauma, and respiratory failure; Level 2 includes chest pain, stroke signs, and severe allergic reactions. It explained that Levels 1-2 require immediate intervention while Levels 3-5 can be assessed by other systems.

Value I Added: I independently selected and defined 14 critical symptom patterns for the safety layer, mapped each to the appropriate CTAS level and emergency action, and designed the match_type logic (any vs all) to handle both single-symptom triggers and multi-symptom combinations based on clinical reasoning.

Prompt Session 11: Python Dictionary Pattern Matching for Rule-Based Systems

Date: February 8, 2026 | Tool: Claude (Anthropic)

Context: While implementing the CTAS rules engine in rules.py, I needed to understand the most efficient way to match user symptoms against my defined rule patterns using Python data structures.

My Prompt: "What is the best Python pattern for matching a list of user-provided strings against a dictionary of keyword rules? I need partial matching so that 'chest pain on left side' matches the keyword 'chest pain'. Should I use a list of dictionaries or a different structure?"

Summary of Response: Claude explained using Python's 'in' operator for substring matching within loops, and recommended a list of dictionaries where each rule contains keywords, match criteria, and associated metadata. It also suggested processing rules in priority order.

Value I Added: I designed the complete CRITICAL_RULES data structure with 14 rules, implemented the priority ordering (Level 1 before Level 2), and built the check_critical_symptoms() function that stops at the first match for efficiency. I also wrote the test script with 7 test scenarios to validate all rule paths.

Prompt Session 12: Research on Random Forest for Healthcare Triage Classification

Date: February 8, 2026 | Tool: Claude (Anthropic)

Context: As part of Sprint 2C.0 (ML Algorithm Research), my instructor required me to justify the choice of Random Forest for disease classification with academic literature before proceeding with model training.

My Prompt: "Can you help me find academic papers that have used Random Forest for healthcare triage or emergency department classification? I need to justify why Random Forest is appropriate for my symptom-to-disease prediction task."

Summary of Response: Claude pointed me to several relevant studies including Goto et al. (2019) on ML-based clinical outcome prediction and Azeez et al. (2015) on ensemble Random Forest for triage. It explained why Random Forest handles high-dimensional sparse data well.

Value I Added: I independently searched for and read the full papers, wrote the complete literature review section comparing the approaches, and synthesised the findings into a research justification that maps each algorithm (Random Forest, K-Means, Isolation Forest) to its specific purpose in my platform.

Prompt Session 13: Understanding Multi-Hot Encoding for Symptom Features

Date: February 9, 2026 | Tool: Claude (Anthropic)

Context: While designing the feature engineering pipeline for the ML model, I needed to understand how to convert the Kaggle dataset's symptom columns (text in 17 separate columns) into a numerical format suitable for Random Forest classification.

My Prompt: "My training data has diseases with symptoms spread across 17 columns. How do I convert this into a format that a Random Forest classifier can use? What is multi-hot encoding and how does it differ from one-hot encoding?"

Summary of Response: Claude explained that multi-hot encoding creates a binary column for each unique symptom, setting 1 for present and 0 for absent, allowing multiple 1s per row (unlike one-hot which has exactly one 1). It described how this creates a sparse matrix suitable for tree-based classifiers.

Value I Added: I designed and implemented the complete feature engineering pipeline: extracting all 131 unique symptoms from the dataset, creating the multi-hot encoding transformation, and building the urgency_mapping.csv that maps each of the 41 diseases to an urgency level based on my own medical judgment.

Prompt Session 14: Stratified Train-Test Split for Balanced Classification

Date: February 9, 2026 | Tool: Claude (Anthropic)

Context: Before training the ML model, I wanted to ensure the train-test split maintained the balanced class distribution of 120 samples per disease.

My Prompt: "When I split my dataset for training, how do I ensure each disease class is equally represented in both the training and test sets? My dataset has 41 diseases with 120 samples each and I want to keep this balance."

Summary of Response: Claude explained the stratify parameter in sklearn's train_test_split, which ensures proportional representation of each class in both splits. It recommended using the disease label column as the stratification target.

Value I Added: I configured the complete training pipeline including the 80/20 stratified split, Random Forest hyperparameters (100 estimators, random_state=42), and the evaluation metrics. I also independently designed the training notebook structure with exploratory analysis and visualisations.

Prompt Session 15: Interpreting 100% Model Accuracy on Training Data

Date: February 10, 2026 | Tool: Claude (Anthropic)

Context: After training the Random Forest model and seeing 100% test accuracy, I was concerned this might indicate overfitting or a methodological error and needed to understand whether this result was legitimate.

My Prompt: "My Random Forest classifier achieved 100% accuracy on the test set. Is this overfitting? The dataset has 41 diseases with 120 samples each and 131 symptom features. How do I determine if this accuracy is genuine or problematic?"

Summary of Response: Claude explained that 100% accuracy can be legitimate when each class has a distinct feature signature, classes are balanced, and the data is clean. It recommended checking for data leakage, examining the confusion matrix, and considering how the model would perform on real-world partial symptom inputs.

Value I Added: I analysed the dataset characteristics independently to confirm the result: verified distinct symptom fingerprints per disease, confirmed no data leakage between train and test sets, and wrote the critical analysis section explaining why real-world accuracy would be lower. I then designed the 70% confidence threshold as a safety mechanism.

Prompt Session 16: Designing a Confidence Threshold for ML Fallback

Date: February 10, 2026 | Tool: Claude (Anthropic)

Context: I needed to design a mechanism where the ML model defers to OpenAI when it is not confident enough in its prediction, particularly for symptoms outside the 41 training categories.

My Prompt: "How does predict_proba work in scikit-learn's Random Forest? I want to set a confidence threshold where if the model's top prediction probability is below a certain percentage, the system falls back to a different recommendation path."

Summary of Response: Claude explained that predict_proba returns probability distributions across all classes, and the maximum probability indicates the model's confidence. It suggested comparing max probability against a threshold and routing low-confidence cases to an alternative path.

Value I Added: I independently designed the 70% threshold value through testing across multiple urgency levels, implemented the fallback routing logic in the chatbot pipeline, and validated the threshold with edge cases including out-of-distribution symptoms that the model was never trained on.

Prompt Session 17: Serializing ML Models with Joblib

Date: February 10, 2026 | Tool: Claude (Anthropic)

Context: After training the model, I needed to save it so the chatbot could load it at runtime without retraining every time the application starts.

My Prompt: "What is the best way to save a trained scikit-learn model so I can load it later in a different Python script? I've seen references to pickle and joblib — which should I use and why?"

Summary of Response: Claude recommended joblib over pickle for scikit-learn models because it is more efficient with large NumPy arrays (which Random Forest uses internally). It explained the dump/load API and the importance of also saving metadata like feature names and class labels.

Value I Added: I implemented the model serialisation saving both model.pkl and model_metadata.json, designed the metadata structure to include feature names, class labels, and training metrics, and verified the loaded model produces identical predictions to the in-memory trained model.

Prompt Session 18: Streamlit Chat Interface Components

Date: February 11, 2026 | Tool: Claude (Anthropic)

Context: Starting Sprint 2D, I needed to understand Streamlit's chat components to build the conversational UI for the symptom assessment chatbot.

My Prompt: "How do I build a chat interface in Streamlit? I need a message history that persists across reruns, user input at the bottom, and the ability to display different styled messages for the bot's responses."

Summary of Response: Claude explained st.chat_message for displaying messages with role-based avatars, st.chat_input for the text input bar, and using session_state to maintain conversation history across Streamlit reruns.

Value I Added: I designed the complete chat UI including the personalised greeting, emergency warning banner, color-coded urgency display, save assessment functionality, and action buttons. The UI layout decisions and BC-specific health resource integration were entirely my own design.

Prompt Session 19: Integrating Three Classification Layers into a Single Pipeline

Date: February 11, 2026 | Tool: Claude (Anthropic)

Context: The most complex task was connecting the three independently built layers (OpenAI, CTAS rules, ML model) into a single orchestrated pipeline in chatbot.py.

My Prompt: "I have three separate modules: openai_utils.py (symptom extraction), rules.py (emergency detection), and a trained ML model. How should I structure a pipeline function that calls them in sequence, with each layer's output feeding into the next, and with branching logic for emergencies and low confidence?"

Summary of Response: Claude suggested a sequential pipeline pattern with early returns: call Layer 1 first, check for errors, then pass to Layer 2, check for critical emergencies (return immediately if found), then pass to Layer 3 with a confidence check branching to either ML results or OpenAI fallback.

Value I Added: I designed the complete process_assessment function architecture, determined the exact sequencing and error handling at each stage, implemented the branching logic for emergency stops and confidence-based routing, and tested the full pipeline across all urgency scenarios.

Prompt Session 20: Color-Coded UI Display for Urgency Levels

Date: February 12, 2026 | Tool: ChatGPT (OpenAI)

Context: I wanted to display assessment results with distinct visual styling for each urgency level to match my UI mockups.

My Prompt: "In Streamlit, how can I apply different background colours and styling to display health urgency results? I need red for Emergency, orange for Urgent, yellow for Routine, and green for Self-Care, similar to a hospital triage colour system."

Summary of Response: ChatGPT explained using Streamlit's st.markdown with custom HTML/CSS for colored badges and containers, and showed how to conditionally apply different styles based on the urgency classification.

Value I Added: I designed the complete visual system matching CTAS colour conventions, created the urgency level guide legend, integrated the BC Health Resources section with HealthLink BC 811 information, and ensured all styling was consistent with the platform's visual identity.

Prompt Session 21: Edge Case Testing Strategy for Healthcare Applications

Date: February 12, 2026 | Tool: Claude (Anthropic)

Context: During integration testing in Sprint 2D, I needed guidance on what edge cases to test for a healthcare symptom assessment system beyond standard functional tests.

My Prompt: "What edge cases should I test for a healthcare chatbot that classifies symptoms? I want to make sure the system handles unusual inputs safely, not just the standard happy-path scenarios."

Summary of Response: Claude suggested testing: non-health inputs (weather questions), single vague symptoms, symptoms outside the training categories, mixed emergency and non-emergency symptoms, and very long or very short inputs. It emphasised that healthcare systems should fail safely.

Value I Added: I designed and executed the complete test plan with 12+ test scenarios covering all urgency levels, non-symptom inputs, edge cases outside the 41 categories, and the confidence threshold fallback. I documented all results and identified the single-turn conversation limitation as a future enhancement.